

SecureShare — Final Capstone Project Report

Comprehensive Academic & Technical Report on Secure File Sharing & DevSecOps Validation

Course: Cybersecurity & Secure Software Engineering Capstone | Date: October 2026

Authors: Sushmitha Reddy, Rahul Kumar, Priya Sharma, Arjun Rao

Executive Summary & Abstract

SecureShare is an enterprise-grade cloud-native secure file sharing platform engineered with Defense-in-Depth principles and continuous automated security validation. Developed in compliance with the IEEE 29148:2018 requirements engineering standard, the system combines Argon2id memory-hard password derivation, RFC 6238 TOTP multi-factor authentication, a 6-stage malware scanning engine, AES-256-GCM envelope encryption, and a tamper-evident SHA-256 blockchain audit ledger. The platform operates within containerized Kubernetes microservices protected by a 14-stage DevSecOps CI/CD pipeline incorporating SAST, SCA, Secret Scanning, DAST, and fuzz testing.

Chapter 1: Problem Definition & Academic Scope

1.1 The Threat Landscape of Modern File Sharing:

Contemporary web file sharing architectures are consistently vulnerable to Broken Object Level Authorization (BOLA), malware ingestion, ransomware distribution, and insider audit tampering.

1.2 Project Objectives:

- Objective 1: Eliminate BOLA vulnerabilities through multi-tenant relationship queries.
- Objective 2: Intercept malware before disk storage via 6-stage stream scanning.
- Objective 3: Ensure non-repudiation and insider threat resistance via SHA-256 blockchain audit chaining.
- Objective 4: Implement continuous automated security verification across the entire SDLC.

Chapter 2: Literature Review & Security Standards

2.1 IEEE 29148 Standard for Requirements Engineering:

Provides systematic structure for stakeholder requirements, system constraints, and verification traceability.

2.2 NIST Cybersecurity Framework & SP 800-53:

Guides identification, protection, detection, response, and recovery controls.

2.3 OWASP Top 10 (2021) Web Security:

Comprehensive mitigations for A01 (Broken Access Control), A02 (Cryptographic Failures), A03 (Injection), A04 (Insecure Design), A05 (Security Misconfig), A07 (Auth Failures), and A09 (Logging Failures).

Chapter 3: STRIDE Threat Modeling & Risk Assessment

3.1 STRIDE Threat Matrix:

Systematic analysis of Spoofing (S-01, S-02), Tampering (T-01, T-02), Repudiation (R-01), Information Disclosure (I-01 to I-03), Denial of Service (D-01, D-02), and Elevation of Privilege (E-01, E-02).

3.2 5x5 Cyber Risk Assessment Matrix:

Quantitative risk scoring (Likelihood x Impact) demonstrating the mitigation of all Inherent Critical risks down to Residual Low risk tiers.

3.3 Attack Tree Decomposition:

Analysis of primary threat vectors targeting confidentiality, integrity, and availability.

Chapter 4: System Architecture & Security Boundaries

4.1 Multi-Tier Trust Boundary Architecture:

Segmentation into Zone 0 (Untrusted Client), Zone 1 (DMZ / Nginx), Zone 2 (Secure App Tier), and Zone 3 (Isolated Data Tier).

4.2 Data Flow Diagrams (DFD Level 0 & Level 1):

Detailed tracking of sensitive file streams, authentication tokens, and audit verification hashes.

4.3 Entity-Relationship Schema Design:

Relational schema linking users, folders, files, versions, permissions, share links, and blockchain audit logs.

Chapter 5: Core Cryptographic & Security Subsystems

5.1 Argon2id Key Derivation:

Configured with memory cost $m=64\text{MB}$, time cost $t=3$ iterations, and parallelism $p=4$ to prevent GPU brute-force attacks.

5.2 AES-256-GCM Envelope Encryption:

Authenticated symmetric encryption with 96-bit unique IV generated per file via `os.urandom(12)` and 128-bit integrity tag.

5.3 Cryptographic Hash-Chained Audit Ledger:

Each audit row computes `sha256(prev_hash + actor_id + action + timestamp)` creating an immutable blockchain structure.

Chapter 6: 6-Stage Malware Scanning & Quarantine Engine

6.1 Stage Decomposition:

- Stage 1: 50MB payload ceiling check.
- Stage 2: libmagic MIME header verification.
- Stage 3: Executable binary (PE/ELF/Mach-O) and script detection.
- Stage 4: Zip bomb recursion and compression ratio analysis.
- Stage 5: ClamAV daemon antivirus signature scanning.
- Stage 6: Shannon entropy calculation to detect obfuscated payloads.

6.2 Air-Gapped Quarantine Isolation:

Infected files moved to `/storage/quarantine` with POSIX 0600 permissions; downloads return HTTP 403 Forbidden.

Chapter 7: Agile Scrum Implementation & Metrics

7.1 Sprint 1 (Core Auth & Scanner): 36 Story Points committed, 36 delivered (100% velocity).

7.2 Sprint 2 (Cryptography & Auditing): 38 Story Points committed, 38 delivered (100% velocity).

7.3 Sprint 3 (DevSecOps & UI): 40 Story Points committed, 40 delivered (100% velocity).

7.4 Zero Carryover Delivery: All 114 story points across 18 user stories completed successfully.

Chapter 8: DevSecOps CI/CD Pipeline & Automated Security Gates

8.1 GitHub Actions Workflow:

14 automated pipeline stages executing on every push and pull request.

8.2 Security Toolchain Integration:

- SAST: Bandit static analysis enforcing zero High/Medium security alerts.
- SCA: Safety and pip-audit checking CVEs in third-party Python packages.
- Secrets: TruffleHog scanning git commit histories for exposed credentials.
- DAST: OWASP ZAP baseline scan performing authenticated vulnerability testing.
- Fuzzing: 500+ malformed input payloads tested against auth and upload endpoints.

Chapter 9: Verification, Quality Assurance & Test Results

9.1 Automated Test Suites:

48 comprehensive automated tests passing with 100% success rate:

- Unit and Integration Suite: 18 passed.
- Security & Threat Mitigation Suite: 11 passed.
- Malformed Input Fuzzing Suite: 19 passed.

9.2 Performance Benchmarking:

AES-256-GCM throughput exceeding 45 MB/s; decryption streaming latency under 95ms.

Chapter 10: Conclusion, Academic Outcomes & Future Work

10.1 Fulfillment of Course & Industry Outcomes:

The SecureShare capstone project successfully demonstrates complete compliance with IEEE 29148 requirements engineering, ISO 27001 control mapping, and real-world DevSecOps security automation.

10.2 Future Roadmap:

- Integration of Hardware Security Modules (HSM) via PKCS#11 for root MEK storage.
- Post-quantum cryptography integration (Kyber-1024 for key exchange).
- Distributed multi-region S3-compatible encrypted object storage backend.